

Integrating the agent into the Oblsk platform

The platform (rockcorp/oblsk) already owns the whole negotiation pipeline: Gmail-backed threads (`negotiationConversation`, `negotiationMessages`), message polling and classification, the review/approval flow, and sending. The one thing it asks a negotiator for is a suggestion: *given this thread, what should the next reply be?* Today that comes from `generateNegotiatorSuggestion()` in `src/modules/negotiation/negotiator/generator.ts`.

This agent is that call, as an HTTP service. Same input shape, same output shape, so the swap is one `fetch` inside the existing Convex node action — no schema changes, no new tables, and humans keep the same review UI.

How it works once integrated (the whole loop)

For each creator, once per inbound message:

1. **A creator emails back.** The platform's Gmail poller adds the message to the `negotiationConversation` thread (this already happens today).
2. **The platform calls the agent** with the thread so far plus the creator's view numbers — `POST /suggest` (one `fetch`, replacing the current `generateNegotiatorSuggestion` call).
3. **The agent reads, prices, and drafts.** It interprets the latest message (intent + any dollar ask), prices the next move off the ladder (open at the anchor, revise to target, never cross walk-away — all from the calculator), and drafts the reply. It returns the message, the move, and whether a human needs to approve or act.
4. **The platform shows it in the review UI** (also already built). A teammate approves → the existing sender emails it. Or the agent flagged `requiresApproval: false` and it auto-sends, within the campaign's dollar ceiling.
5. **Repeat** until the agent's move is `accept` (deal closed), `escalate` (handed to a person), or it asks a human for something it can't answer.

The agent holds no state of its own — the Gmail thread is the record. That is why a person can jump into a thread at any point and the agent picks right back up (see §3). To watch this loop by hand before wiring it in, run `py demo.py --chat`: you type the creator's messages, the agent runs the exact same `read` → `price` → `draft` it runs in step 3.

1. Run the service

```
pip install -r requirements.txt anthropic
set ANTHROPIC_API_KEY=...           # Claude drafts + reads; omit for templates
set OBLSK_SERVICE_TOKEN=<random>    # optional but recommended: bearer auth
py -m oblsk_negotiator.service --campaign examples/unest_campaign.yaml --port
8788
```

`GET /health` returns `{ok, campaign, llm}`. One service instance serves one campaign config; run one per campaign (they're cheap) or front several with a router if that ever matters.

2. The contract

POST /suggest takes the fields the platform already assembles for its `NegotiatorInput`:

```
{
  "creatorHandle": "@thecollinskidsfamily",      // optional, used in drafting
  "medianViewsPerVideo": 150000,                 // fallback when few posts
  "recentPostViews": [162000, 98000, ...],       // preferred: >=6 real posts
  "messages": [                                  // the whole thread, any order
    { "role": "creator", "receivedAt": 1718713800000, "content": "..."},
    { "role": "brand", "receivedAt": 1718810460000, "content": "..."}
  ]
}
```

The response is the platform's `NegotiatorSuggestion` shape plus an agent block with the full-fidelity decision:

```
{
  "message": "Hi Albert, ...",                  // the drafted reply
  "strategy": {
    "kind": "package_counter",                  // nearest platform kind
    "title": "Counter with a package",
    "instructions": "Flat rejected ($48,000). Offer a 3-format package ...",
    "packageTerms": { "videoCount": 3, "totalUsd": 7000,
                      "effectivePerVideoUsd": 2333.33, "requestedVideoCount": 1},
    "cpmTerms": null,
    "debug": { "creatorAsk": {...}, "variance": {...},
               "targetPerVideoUsd": 2043.0, "maxPerVideoUsd": 2860.0, ...}
  },
  "model": "claude-opus-4-8",                  // null when offline
  "agent": {
    "action": "escalate_bundle",                // the precise move
    "requiresApproval": true,
    "humanPrompt": null,                       // set when a person must act
    "deal": {...},
    "ladder": { "anchorUsd": 1550, "targetUsd": 2043, "walkAwayUsd": 2860,
                "binding": "downside-ROI", "downsideLimited": false}
  }
}
```

`strategy.kind` maps into the platform's closed union (it only drives UI display):

open/hold/answer → `initial_offer`, bundle/sweetener → `package_counter`, accept → `accept`, hand-offs → `escalate`, and the human-in-the-loop pauses (`ask_human`, `propose_call`) → `missing_context` — those two always carry `agent.humanPrompt` saying exactly what a person needs to do. Errors return 400 (bad payload) with `{"error": "..."}.`

3. Statelessness (why there is no agent database)

The platform's thread is the source of truth, so the service keeps no state: every request carries the whole thread, and negotiation state is derived from it. Dollar offers found in `brand` messages become the agent's recorded positions — whoever sent them, the agent or a person — so the agent picks up mid-flight threads without re-opening at the anchor, and humans can freely take over and hand back. The creator's first ask is read from their messages the same way.

4. Convex-side wiring

In the "use node" action that currently calls the generator (`src/convex/negotiation/actions.ts`), swap in a fetch:

```
// alongside generateNegotiatorSuggestion(input)
async function fetchAgentSuggestion(input: NegotiatorInput) {
  const res = await fetch(`${process.env.NEGOTIATOR_URL}/suggest`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${process.env.NEGOTIATOR_TOKEN}`,
    },
    body: JSON.stringify({
      creatorHandle: input.creatorHandle,
      medianViewsPerVideo: input.medianViewsPerVideo,
      recentPostViews: input.recentPostViews,
      messages: input.messages,
    }),
  });
  if (!res.ok) throw new Error(`negotiator: ${res.status} ${await res.text()}`);
  return (await res.json()) as NegotiatorSuggestion & {
    agent: {
      action: string;
      requiresApproval: boolean;
      humanPrompt: string | null;
    };
  };
}
```

Set `NEGOTIATOR_URL` and `NEGOTIATOR_TOKEN` in the Convex deployment env. Recommended handling of the extension block:

- `agent.requiresApproval` → keep the conversation in the existing `needs_human_confirmation` status instead of auto-sending.
- `agent.humanPrompt` non-null → surface it in the inbox UI; for `ask_human` nothing should be sent until a teammate answers (add the answer to the thread and re-request a suggestion); for `propose_call` the drafted message can go out, and a teammate owns the invite and the call.
- `agent.action === "escalate_human"` → assign the thread to a person (contract-terms negotiation always lands here by design).

5. What the platform keeps doing

Gmail polling, message classification, deal-terms extraction, sending, and the approval UI all stay platform-side. The agent only decides and drafts. Pricing inputs (`conversion_rate`, LTV, ROI targets, call windows, the brief) live in the campaign YAML the service was started with — calibrate those per campaign; `docs/CALCULATOR.md` explains every number the ladder produces.